
UNTANGLING EQUIFINALITY: A LARGE-SAMPLE BENCHMARK OF MACHINE-LEARNING MODELS FOR RAINFALL–RUNOFF FORECASTING IN EURASIAN BASINS

A PREPRINT

Khuzin Eldar
Chair of Data Analysis
Moscow Institute of Physics and Technology
Moscow, Russia
khuzin.er@phystech.su

Novikov Ivan
Affiliation
Address
email

Abramov Dmitrii
Affiliation
Address
email

ABSTRACT

In this article, we address the problem of equifinality in data-driven hydrological modeling. Equifinality is when different feature sets or model architectures yield similar predictive efficiency. We investigate the influence of static catchment features on predictions made by traditional machine learning models (e.g., random forest, gradient boosting), recurrent neural networks across various river basins in Eurasia. By analyzing feature importance and model behavior, we identify dominant predictors and assess the practicality of employing complex models in terms of interpretability and computational cost. This approach provides insights into the trade-offs between model complexity and predictive robustness in the presence of equifinality.

Keywords Flood discharge · Rainfall–Runoff Modeling · Equifinality · Machine Learning · SHAP · Static Catchment Characteristics

List of suggested changes

■ describe the machine 8

1 Introduction

Accurate rainfall–runoff forecasting is essential for water–resource management (long–term water–resource planning, reservoir operation) and flood risk assessment. Nevertheless, forecast skill in the calibration period does not automatically translate into reliable predictions for unseen basins or extreme events; rigorous tests of model generalisation are therefore indispensable.

Traditionally, physically based hydrological models are calibrated separately for each basin; yet it is often observed that markedly different parameter sets or model structures can reproduce the same hydrograph with near–identical accuracy — a phenomenon known as *equifinality* (literally “equal finality”) Beven and Binley 1992; Freer, Beven, and Ambroise 1996. Equifinality undermines the notion of a unique “best” model and complicates any attempt to ascribe physical meaning to fitted parameters.

Machine–learning (ML) methods intensify this challenge. The flexibility of modern data–driven methods — from tree ensembles to deep neural networks — gives them many “degrees of freedom” to fit data and can exploit subtle correlations that lack causal significance. High predictive skill alone therefore *does not guarantee* that an ML model has learned the underlying hydrological processes. When several equifinal models agree on historical data, some may break down under non-stationary climate or in ungauged basins.

Static catchment descriptors — such as drainage area, mean slope, soil texture, and aridity index—offer a partial remedy by injecting physical context into data–driven models; incorporating them into an LSTM has been shown to

yield a “universal” regional model that generalises across basins Kratzert et al. 2019. Yet under equifinality different architectures may leverage the same descriptors in different ways or ignore them altogether while still achieving comparable accuracy. Determining which inputs truly drive predictions is thus a prerequisite for trustworthy operational use.

This study disentangles equifinality in data-driven rainfall–runoff modeling through a controlled, large-sample comparison of three representative ML architectures: a random forest, a gradient boosting and a long short-term memory (LSTM) network. All models receive identical meteorological time series and static basin attributes, eliminating data inequality as a confounder. We evaluate their predictive skill — quantified by Nash–Sutcliffe efficiency (NSE), root mean square error (RMSE), and mean absolute percentage error (MAPE) — on 1,073 Eurasian basins and inspect feature contributions via SHAP values.

Our contributions are three-fold:

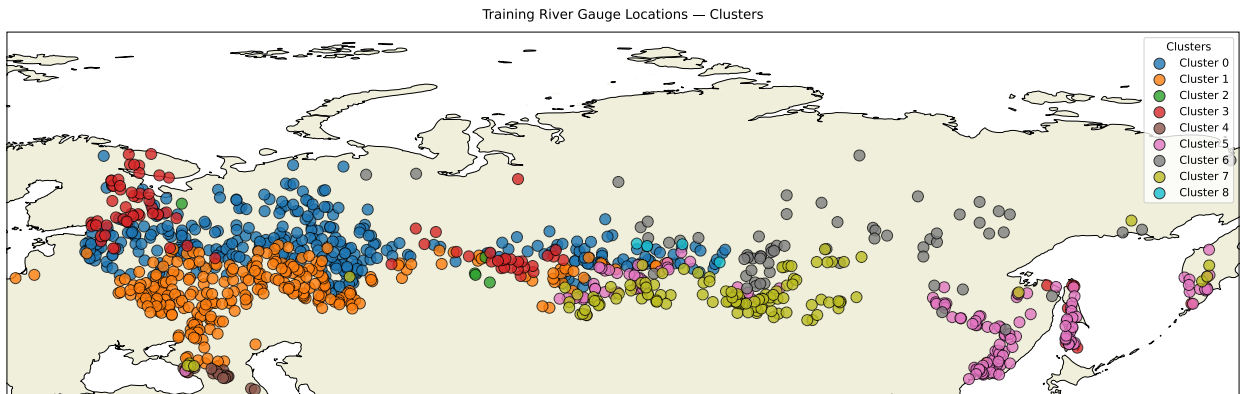
1. We quantify how often alternative architectures yield statistically indistinguishable performance, providing a basin-wise map of equifinality.
2. We reveal the degree to which equifinal models rely on the same (or different) static and dynamic predictors.
3. We discuss the trade-off between accuracy, interpretability, and computational cost, offering guidance for practitioners choosing between simple and complex ML tools.

The remainder of the paper is organised as follows. Section 2 describes the data set and study area. Section 3 details the models, training procedure, loss functions, and evaluation metrics. Section 4 describes the experiments. Section 5 presents the experimental results, including an analysis of equifinality and feature importance. Section 6 discusses the implications for hydrological theory and operational forecasting. Finally, Section ?? summarises the key findings and outlines future work.

2 Data and Study Area

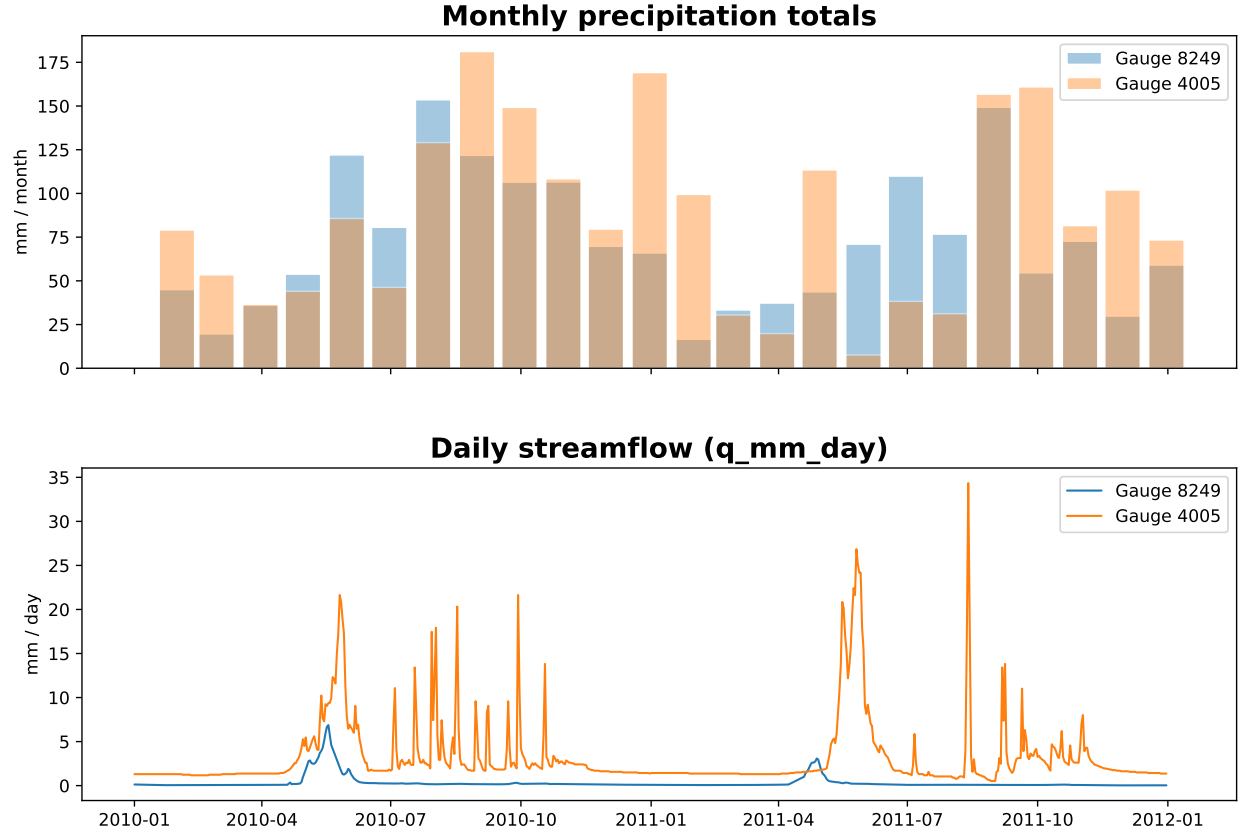
2.1 Study Area

This study focuses on river basins across Eurasia, located approximately between latitudes 41.42–71.97 and longitudes 19.92–170.50. The selected region encompasses a wide range of static basin characteristics and climatic conditions, making it suitable for analyzing hydrological equifinality.



2.2 Hydrometeorological Data

The dataset includes daily time series from 2008 to 2022 for the following features: precipitation (prcp), minimum temperature (t_min), maximum temperature (t_max), mean temperature (t_mean), and river discharge (q_mm_day).



2.3 Static Basin Attributes

Static (time-invariant) characteristics of the river basins were derived from the HydroSHEDS database Linke et al. 2019. A total of 15 features¹ were selected based on two main criteria: (i) low pairwise correlation and (ii) high interpretability for ability to explain the models.

In total, data were available for 2,201 river basins. For model training and subsequent analysis, we selected a subset of 1,073 basins – basins without missing values to ensure data quality and consistency across samples.

ID	Attribute	Column Name
L07	Forest Cover Extent	for_pc_sse
L08	Cropland Extent	crp_pc_sse
L09	Pasture Extent	pst_pc_sse
L10	Irrigated Area Extent	ire_pc_sse
A03	Urban Extent	urb_pc_sse
S07	Karst Area Extent	kar_pc_sse
P01	Elevation	ele_mt_sav
P02	Terrain Slope	slp_dg_sav
C06	Aridity Index	ari_ix_sav
H10	Groundwater Table Depth	gwt_cm_sav
S05	Soil Water Content (Annual Avg)	swc_pc_syr
S01	Clay Fraction in Soil	cly_pc_sav
A06	Human Footprint Index	hft_ix_s09
P04	Latitude	lat
P05	Longitude	lon

Table 1: List of selected static features and their corresponding column names.

3 Methods

3.1 Problem Statement and Notation

We address a regression problem where we aim to predict a target from a set of features.

Notation

- Let $y \in \mathbb{R}^n$ and $y_i \in \mathbb{R}$ denote the target; $x \in \mathbb{R}^{n \times d}$ and $x_i \in \mathbb{R}^d$ denote d features associated with y_i , where n is the number of samples;
- Let the dataset be $\mathcal{D} = \{(x_i, y_i) \mid i \in \overline{1, n}\}$ – set of n pairs for features and targets.
- Let the models class be denoted as $\mathcal{F}$;
- Let $\hat{y}_i := f(x_i)$, $f \in \mathcal{F}$ denotes estimation of y_i based on x_i ;
- Let assume $\hat{y} := (\hat{y}_1, \dots, \hat{y}_n) = (f(x_1), \dots, f(x_n))$.

3.2 Optimization statement

In general, in each experiment we find the optimal model f^* s.t.

$$f^* = \arg \min_{f \in \mathcal{F}} (\mathcal{L}(f, \mathcal{D}) + \Omega(f)), \quad (1)$$

where $\mathcal{L}(f, \mathcal{D})$ denotes the empirical loss of model f on dataset $\mathcal{D}$, and $\Omega(f)$ is a regularization term penalizing model complexity.

Let $\ell(\cdot, \cdot)$ denotes the actual loss function, so $\mathcal{L}(f, \mathcal{D}) = \ell(y, \hat{y}) = \frac{1}{n} \sum_{i=1}^n \ell(y_i, \hat{y}_i)$.

In our work we choose ℓ as a mean absolute percentage error (MAPE), so 1 can be rewritten as

$$f^* = \arg \min_{f \in \mathcal{F}} \left(\sum_{i=1}^n \left| \frac{y_i - f(x_i)}{y_i} \right| + \Omega(f) \right). \quad (\text{Opt. task})$$

3.2.1 Pseudo-MAPE Objective

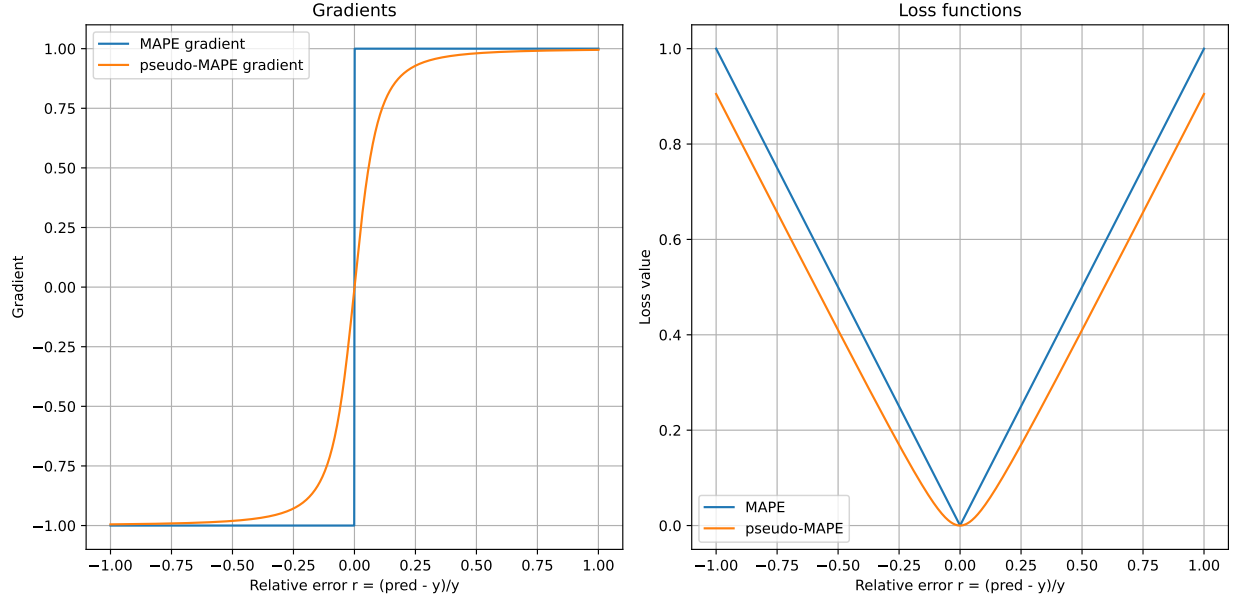
But standard MAPE is non-differentiable at $y_i = \hat{y}_i$ and yields unbounded gradients as $y_i \rightarrow 0$. To obtain smoother, more stable updates in gradient-based learners, we define a pseudo-MAPE loss with smoothing parameter $\delta > 0$:

$$L_\delta(r) = \sqrt{r^2 + \delta^2}, \quad r_i = \frac{\hat{y}_i - y_i}{y_i}.$$

Its gradient and Hessian (needed for XGBoost’s custom objective) are

$$g_i = \frac{r_i}{y_i \sqrt{r_i^2 + \delta^2}}, \quad h_i = \frac{\delta^2}{y_i^2 (r_i^2 + \delta^2)^{3/2}}.$$

In our experiments we set $\delta = 10^{-1}$ to balance smoothness and closeness to true MAPE.



3.3 Model's descriptions

3.3.1 Regression trees

In experiments we use regression trees as base predictors. In general, regression tree is binary tree, which partitions the feature space into regions and predicts constant values within each region, formally:

$$f(x) = \sum_{l=1}^L \mathbf{1}_{\{x \in R_l\}} \cdot \omega_l, \quad (2)$$

where:

- R_l , $l \in \overline{1, L}$ form a partition of $\mathbb{R}^d$ into disjoint sets;
- ω_l are constant weights on tree leaves.

To perform the partitioning into R_l , each node of the binary tree acts as a simple predicate of the form $x^{(j)} \leq t$, used to select between the left and right child. Here, $x^{(j)}$ denotes a coordinate in the features $\mathbb{R}^d$ space, and t is a *threshold*.

To select a threshold t and the corresponding feature index j at each node, the algorithm maximizes an objective function known as the *information gain* (IG , $\mathcal{IG}$). We are going to use the same IG as Chen and Guestrin 2016. Let assume $\hat{y}_i^{(0)}$ as initial prediction (constant):

$$\mathcal{IG}(t, j) = \frac{1}{2} \left[\frac{(\sum_{i \in \mathcal{I}_L} g_i)^2}{\sum_{i \in \mathcal{I}_L} h_i + \lambda} + \frac{(\sum_{i \in \mathcal{I}_R} g_i)^2}{\sum_{i \in \mathcal{I}_R} h_i + \lambda} - \frac{(\sum_{i \in \mathcal{I}} g_i)^2}{\sum_{i \in \mathcal{I}} h_i + \lambda} \right] - \gamma,$$

where:

- $\mathcal{I}_L = \{i : x_i^{(j)} \leq t\}$ – indices, which are going to be in the left child if node use threshold t and feature j ;
- $\mathcal{I}_R = \{i : x_i^{(j)} > t\}$ – same, but in the right child;
- $g_i = \frac{\partial \ell(y_i, \hat{y}_i)}{\partial \hat{y}_i} \Big|_{\hat{y}_i = \hat{y}_i^{(0)}}$ and $h_i = \frac{\partial^2 \ell(y_i, \hat{y}_i)}{\partial \hat{y}_i^2} \Big|_{\hat{y}_i = \hat{y}_i^{(0)}}$ are the first and second derivatives of the loss;
- λ is the L_2 regularization weight on leaf scores;
- γ is the complexity penalty for adding a new leaf.

In other words, we find a best split in node for Opt. task where complexity penalizes is $\Omega(f) = \gamma L + \frac{1}{2} \lambda \|w\|_2^2$

Then, once the tree structure is fixed, we assign each leaf l the weight

$$\omega_l = -\frac{G_l}{H_l + \lambda}, \quad \text{where } G_l = \sum_{i \in R_l} g_i, \quad H_l = \sum_{i \in R_l} h_i.$$

3.3.2 Random Forest

Random Forest (RF) is an ensemble of B regression trees Breiman 2001, each trained on a bootstrap draw of the training set. At every split, instead of scanning all d features, only a random subset of $m_{\text{try}} \ll d$ candidates is used. Thus, each tree $f^{(i)}$ is both bagged and feature-subsampled, and the RF prediction is

$$f(x) = \frac{1}{B} \sum_{I=1}^B f^{(i)}(x). \quad (3)$$

This dual randomness (bagging + feature subsampling) (1) lowers variance by averaging many decorrelated trees, and (2) secure from overfitting despite fully grown leaves.

Primary hyperparameters:

- B – number of trees;
- m_{try} – (features per split);
- “Bootstrap sample size” – n draws with replacement.

Breiman 2001 showed that as $B \rightarrow \infty$, RF variance go to zero if trees correlation is lowest.

3.3.3 Gradient Boosting

Gradient Boosting Friedman 2001 builds an additive ensemble of B regression trees by successive error correction. Starting from an initial guess $f^{(0)}(x)$, each new tree $h^{(b)}$ is fit to the negative gradient (residuals) of the loss ℓ with respect to the current model. Formally, GBM is

$$f_{(B)}(x) = f^{(0)} + \nu \sum_{i=1}^B f^{(i)}(x), \quad (4)$$

where

- ν – is a learning rate hyperparameter;
- B – number of trees;
- $f^{(b)}$ – regression tree;

and each of the tree is trained to predict the errors of the previous state model $f_{(B-1)}$:

$$f^{(B)} = \arg \min_{f \in \mathcal{F}} \sum_{i=1}^n \ell(y_i - f_{(B-1)}(x_i), f(x_i)) \quad (5)$$

3.4 Evaluation Metrics

We evaluate models using RMSE, MAE, MAPE, and Nash–Sutcliffe Efficiency (NSE):

Root Mean Squared Error (RMSE)

$$\text{RMSE}(y, \hat{y}) = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}. \quad (\text{RMSE})$$

Mean Absolute Error (MAE)

$$\text{MAE}(y, \hat{y}) = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|. \quad (\text{MAE})$$

Mean Absolute Percentage Error (MAPE)

$$\text{MAPE}(y, \hat{y}) = \frac{1}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right| \cdot 100\%. \quad (\text{MAPE})$$

Nash-Sutcliffe Efficiency (NSE) Originally developed in hydrology, the NSE evaluates the predictive power of hydrological models.

$$\text{NSE}(y, \hat{y}) = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}. \quad (\text{NSE})$$

Interpretation:

- NSE = 1 indicates a perfect match between model and observations.
- NSE = 0 implies that the model is no better than using the mean of the observed data.
- NSE < 0 suggests the model predictions are worse than the mean.

3.5 Feature Importance Analysis (SHAP Analysis)

To interpret the predictions and identify the key drivers influencing model performance, we utilize SHapley Additive exPlanations (SHAP) Lundberg and Lee 2017. SHAP values quantify the contribution of each feature to the model’s predictions based on principles from game theory. Specifically, for a feature j and prediction $f(x)$, SHAP computes the contribution as:

$$\phi_j = \sum_{S \subseteq \mathcal{F} \setminus \{j\}} \frac{|S|!(|\mathcal{F}| - |S| - 1)!}{|\mathcal{F}|!} [f_{S \cup \{j\}}(x_{S \cup \{j\}}) - f_S(x_S)], \quad (6)$$

where $\mathcal{F}$ is the set of all features, and S is a subset of features excluding feature j . The idea is to evaluate the contribution to the model’s predictive performance by removing particular features.

3.6 Correlation Analysis of Static Features

To identify a subset of static features that are both informative and mutually independent, we perform a Pearson correlation analysis exclusively on the static feature set:

$$r_{X,Y} = \frac{\text{cov}(X, Y)}{\sigma_X \sigma_Y} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}}. \quad (7)$$

We compute all pairwise correlations $r_{i,j}$ among static features and then retain only those features whose absolute mutual correlation satisfies $|r_{i,j}| < 0.7$. By selecting this subset of least-correlated static variables, we further select only those features that are readily explainable and whose physical interpretation — such as basin area, mean slope, or soil type — is clear. This two-step selection ensures that the final static features both distinguish among river catchments and remain interpretable for insight into hydrological processes.

4 Computational experiment

Software

All experiments were conducted in Python 3.12 using the following open-source libraries:

- `scikit-learn` (version **) for implementation of Random Forest models Pedregosa et al. n.d.
- `XGBoost` (version **) for gradient boosting models Chen and Guestrin 2016.

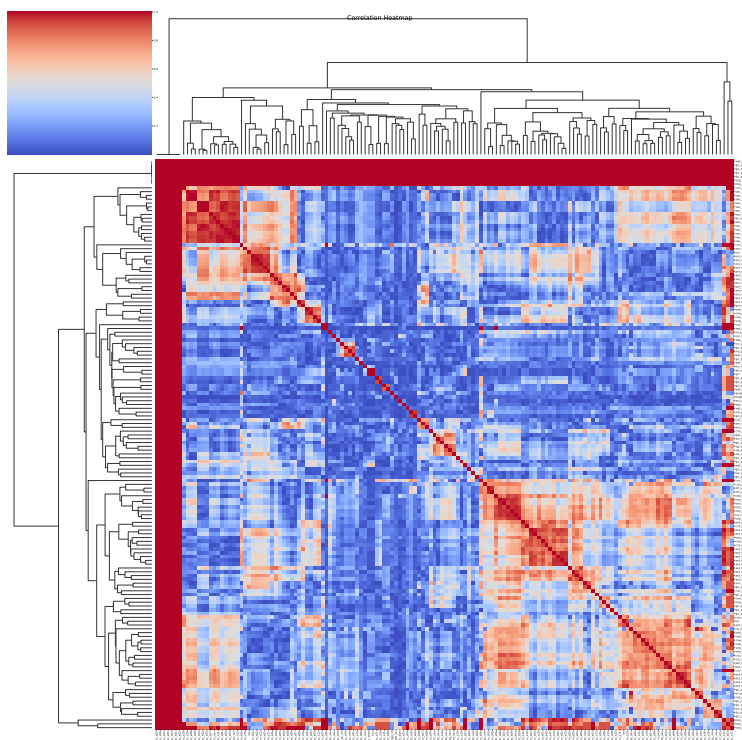
- Dask (version) for parallel and distributed computation Rocklin 2015.
- NumPy and Pandas for data manipulation and preprocessing.
- Matplotlib and Seaborn for visualization.

Experiments were executed on a machine . The complete source code and environment configuration files are available at: <https://github.com/intsystems/2025-Project-188>.

describe the machine

4.1 Correlation analysis

To eliminate model equifinality arising from mathematical similarity among features (which is especially critical for tree-based algorithms), we conducted a correlation analysis^{4.1} using Pearson’s correlation coefficient. From the full feature set, 20 static features and 3 “reserve” features (duplicating the main ones when some data were missing) were selected. For the final feature selection, we first computed the sum of absolute correlations of each feature with all others, chose those with the lowest totals, and then picked the most readily interpretable features from that subset.



4.2 Clusterization

To assess how well the models actually capture hydrological patterns, we grouped river basins into clusters based on static features. After training on the full dataset, we expected the model to assign each basin to its cluster and recognize these groups as significant. We conducted a visual analysis of clustering using the k-means algorithm with prior standard scaling of the features. The number of clusters was varied from 5 to 14, and based on the heatmap that most clearly reflected differences among basins by their static attributes, we decided to use nine clusters.

4.3 Data-Processing Pipeline

1. Split the dataset into training (train) and validation (valid) sets;
2. Perform hyperparameter tuning with Optuna to select the optimal model within each algorithm class;
3. Train the model on the training set;
4. Conduct feature-importance analysis using SHAP for each model class;
5. Compare models based on key metrics on both the training and validation sets.

5 Results

5.1 Experiment 1. Random Forest

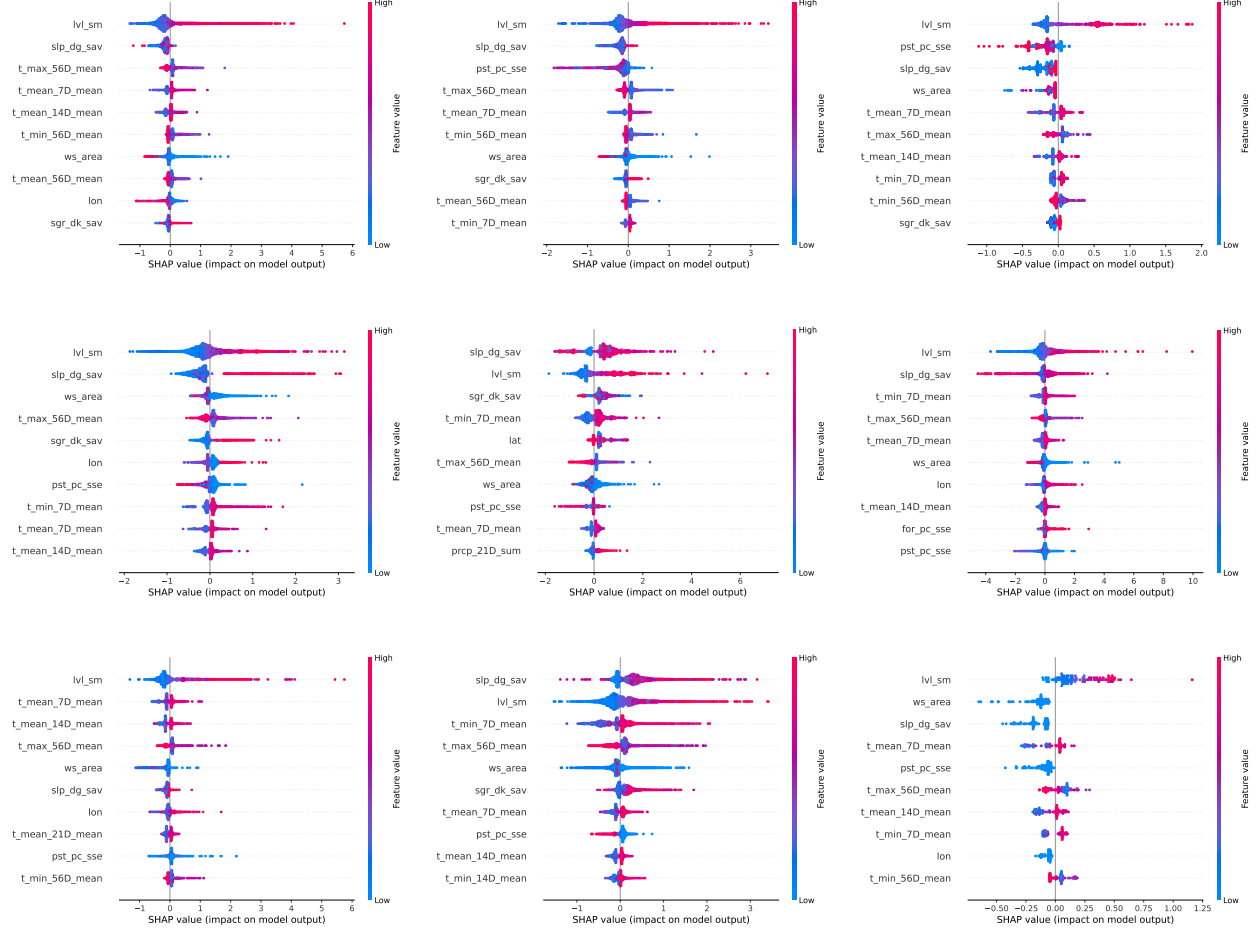


Figure 1: SHAP values for Random forest

Figure 1 shows the SHAP values for the Random Forest model across all nine clusters. Remarkably, soil moisture level (*lvl_sm*) and slope gradient (*slp_dg_sav*) consistently rank highest in importance, while precipitation variability (*pst_pc_sse*) occupies a middle position. This uniform pattern suggests that the model relies mainly on common hydrological drivers rather than on cluster-specific attributes. Such behavior may indicate either insufficient differentiation of clusters by static features or the overwhelming influence of a few key variables in the training data.

5.2 Experiment 2. Gradient boosting

Figure 2 shows the SHAP values for the Gradient Boosting model across all nine clusters. Unlike the Random Forest, we observe substantial variability in top features between clusters. While soil moisture level (*lvl_sm*) still frequently appears in the top three, some clusters are dominated by maximum 56-day temperature (*t_max_56D_mean*, e.g., clusters 0 and 7), others by watershed area (*ws_area*, clusters 2 and 8) or slope gradient (*slp_dg_sav*, cluster 1). Precipitation dynamics (*pst_pc_sse*) rank highly only in a few clusters (1 and 8). This pattern suggests that Gradient Boosting better differentiates hydrological characteristics of individual basin groups, adapting to their specific local drivers.

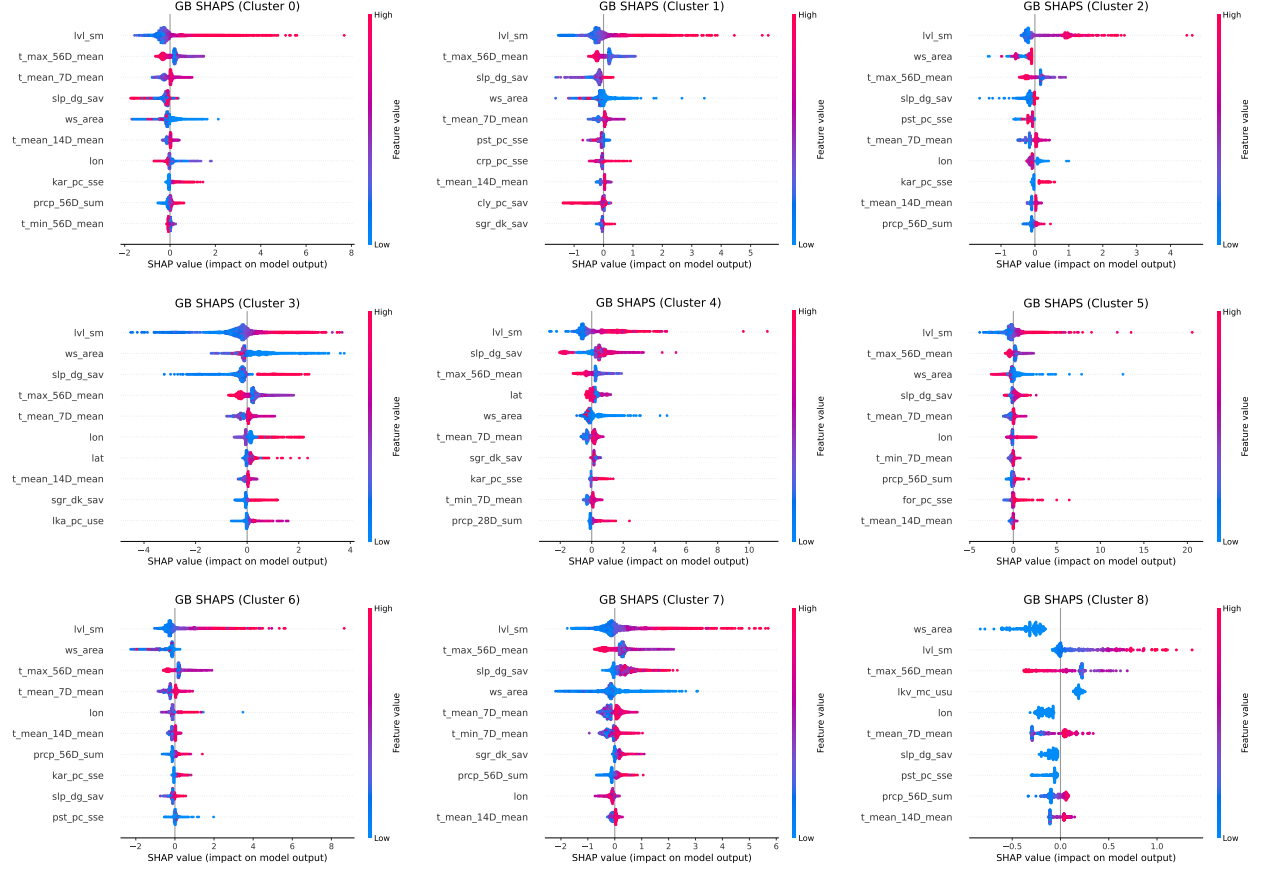


Figure 2: SHAP values for Gradient boosting

5.3 Metrics

As shown in Figure ??, we evaluate model performance using four standard metrics — Nash–Sutcliffe Efficiency (NSE), Root–Mean–Square Error (RMSE), Mean Absolute Error (MAE), and Mean Absolute Percentage Error (MAPE) — for clusters 0–3.

- Overall performance.** Across all clusters, the Random Forest consistently achieves higher NSE values and lower error magnitudes (RMSE, MAE, and MAPE) than the Gradient Boosting model. For example, in cluster 0 RF attains NSE = 0.922 versus GB’s 0.780, while its RMSE (0.343 m³s) is roughly 40.
- Cluster-specific insights.**
 - Cluster 1:* RF (NSE 0.909) greatly outperforms GB (0.634), with GB’s MAPE exceeding 1
 - Cluster 2:* Both models perform well (NSE > 0.8), but RF’s mean errors (RMSE 0.112, MAE 0.052) remain smallest, reflecting accurate predictions even in basins with low discharge variability.
 - Cluster 3:* The performance gap persists, with RF achieving NSE 0.939 and GB 0.829, and RF’s MAE nearly half that of GB.
- Implications.** The superior and more stable skill of the Random Forest — particularly its robustness against outliers (as evidenced by lower RMSE) — suggests it as the preferred modeling approach for these hydrological clusters.

6 Discussion

Our benchmark reveals *equifinality*: Random Forest, Gradient Boosting achieve statistically indistinguishable Nash–Sutcliffe efficiencies on many basins, indicating multiple plausible solutions.

Gradient Boosting					
cluster	NSE	RMSE	MAE	MAPE	Total basins
0	0.780	0.577	0.259	0.655	44 544
1	0.634	0.372	0.143	1.228	38 371
2	0.820	0.214	0.120	0.454	1 096
3	0.829	0.708	0.325	1.093	12 583

Random Forest					
cluster	NSE	RMSE	MAE	MAPE	Total basins
0	0.922	0.343	0.149	0.428	44 544
1	0.909	0.185	0.070	0.590	38 371
2	0.950	0.112	0.052	0.173	1 096
3	0.939	0.424	0.158	0.653	12 583

Figure 3: Result metrics

Feature–importance analyses show that RF consistently emphasizes soil moisture and slope, whereas GB adapts to local characteristics (e.g., temperature regimes, watershed area). This underlines the need for interpretability checks to uncover divergent decision logics.

In practical terms, RF delivers the highest accuracy but incurs greater memory and prediction–latency costs; GB offers a more compact model with marginally lower performance; Future work should integrate extreme–event indicators, human–infrastructure variables, transfer–learning frameworks for ungauged basins, and model–agnostic interpretability methods to validate causal consistency under nonstationary conditions.

References

- Beven, Keith and Andrew Binley (July 1992). “The Future of Distributed Models: Model Calibration and Uncertainty Prediction.” In: *Hydrological Processes* 6.3, pp. 279–298. ISSN: 0885-6087, 1099-1085. DOI: 10.1002/hyp.3360060305. URL: <https://onlinelibrary.wiley.com/doi/10.1002/hyp.3360060305> (visited on 05/16/2025).
- Breiman, Leo (Oct. 1, 2001). “Random Forests.” In: *Machine Learning* 45.1, pp. 5–32. ISSN: 1573-0565. DOI: 10.1023/A:1010933404324. URL: <https://doi.org/10.1023/A:1010933404324> (visited on 04/24/2025).
- Chen, Tianqi and Carlos Guestrin (Aug. 13, 2016). “XGBoost: A Scalable Tree Boosting System.” In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. KDD ’16: The 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. San Francisco California USA: ACM, pp. 785–794. ISBN: 978-1-4503-4232-2. DOI: 10.1145/2939672.2939785. URL: <https://dl.acm.org/doi/10.1145/2939672.2939785> (visited on 04/16/2025).
- Freer, Jim, Keith Beven, and Bruno Ambroise (1996). “Bayesian Estimation of Uncertainty in Runoff Prediction and the Value of Data: An Application of the GLUE Approach.” In: *Water Resources Research* 32.7, pp. 2161–2173. ISSN: 1944-7973. DOI: 10.1029/95WR03723. URL: <https://onlinelibrary.wiley.com/doi/abs/10.1029/95WR03723> (visited on 05/16/2025).
- Friedman, Jerome H. (Oct. 1, 2001). “Greedy Function Approximation: A Gradient Boosting Machine.” In: *Ann. Statist.* 29.5. ISSN: 0090-5364. DOI: 10.1214/aos/1013203451. URL: <https://projecteuclid.org/journals/annals-of-statistics/volume-29/issue-5/Greedy-function-approximation-A-gradient-boosting-machine/10.1214/aos/1013203451.full> (visited on 04/16/2025).
- Kratzert, Frederik et al. (Dec. 17, 2019). “Towards Learning Universal, Regional, and Local Hydrological Behaviors via Machine Learning Applied to Large-Sample Datasets.” In: *Hydrology and Earth System Sciences* 23.12, pp. 5089–5110. ISSN: 1027-5606. DOI: 10.5194/hess-23-5089-2019. URL: <https://hess.copernicus.org/articles/23/5089/2019/> (visited on 05/16/2025).
- Linke, Simon et al. (Dec. 9, 2019). “Global Hydro-Environmental Sub-Basin and River Reach Characteristics at High Spatial Resolution.” In: *Scientific Data* 6.1, p. 283. ISSN: 2052-4463. DOI: 10.1038/s41597-019-0300-6. URL: <https://www.nature.com/articles/s41597-019-0300-6> (visited on 05/17/2025).

- Lundberg, Scott M and Su-In Lee (2017). “A Unified Approach to Interpreting Model Predictions.” In: *Advances in Neural Information Processing Systems*. Vol. 30. Curran Associates, Inc. URL: <https://proceedings.neurips.cc/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html> (visited on 05/12/2025).
- Pedregosa, Fabian et al. (n.d.). “Scikit-Learn: Machine Learning in Python.” In: *MACHINE LEARNING IN PYTHON* ().
- Rocklin, Matthew (2015). “Dask: Parallel Computation with Blocked Algorithms and Task Scheduling.” In: *Python in Science Conference*. Austin, Texas, pp. 126–132. DOI: 10.25080/Majora-7b98e3ed-013. URL: <https://doi.curvenote.com/10.25080/Majora-7b98e3ed-013> (visited on 04/16/2025).